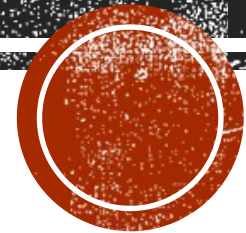


# SOFTWARE ENGINEERING

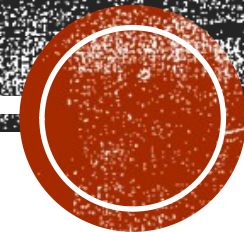


Unit III:  
Estimation and Scheduling

By:  
Mayur S. Chavan  
Assistant Professor  
P.I.C.T. Pune

| Semester-IV  |                                                           |                              |           |          |                              |         |           |           |      |       |               |           |          |       |
|--------------|-----------------------------------------------------------|------------------------------|-----------|----------|------------------------------|---------|-----------|-----------|------|-------|---------------|-----------|----------|-------|
| Course Code  | Course Name                                               | Teaching Scheme (Hours/Week) |           |          | Examination Scheme and Marks |         |           |           |      |       | Credit Scheme |           |          |       |
|              |                                                           | Lecture                      | Practical | Tutorial | Mid-Sem                      | End-Sem | Term work | Practical | Oral | Total | Lecture       | Practical | Tutorial | Total |
| 207003       | <a href="#">Engineering Mathematics III</a>               | 03                           | -         | 01       | 30                           | 70      | 25        | -         | -    | 125   | 03            | --        | 01       | 04    |
| 210252       | <a href="#">Data Structures and Algorithms</a>            | 03                           | -         | -        | 30                           | 70      | -         | -         | -    | 100   | 03            | -         | -        | 03    |
| 210253       | <a href="#">Software Engineering</a>                      | 03                           | -         | -        | 30                           | 70      | -         | -         | -    | 100   | 03            | -         | -        | 03    |
| 210254       | <a href="#">Microprocessor</a>                            | 03                           | -         | -        | 30                           | 70      | -         | -         | -    | 100   | 03            | -         | -        | 03    |
| 210255       | <a href="#">Principles of Programming Languages</a>       | 03                           | -         | -        | 30                           | 70      | -         | -         | -    | 100   | 03            | -         | -        | 03    |
| 210256       | <a href="#">Data Structures and Algorithms Laboratory</a> | -                            | 04        | -        | -                            | -       | 25        | 25        | -    | 50    | -             | 02        | -        | 02    |
| 210257       | <a href="#">Microprocessor Laboratory</a>                 | -                            | 02        | -        | -                            | -       | 25        | -         | 25   | 50    | -             | 01        | -        | 01    |
| 210258       | <a href="#">Project Based Learning II</a>                 | -                            | 04        | -        | -                            | -       | 50        | -         | -    | 50    | -             | 02        | -        | 02    |
| 210259       | <a href="#">Code of Conduct</a>                           | -                            | -         | 01       | -                            | -       | 25        | -         | -    | 25    | -             | -         | 01       | 01    |
| 210260       | <a href="#">Audit Course 4</a>                            |                              |           |          |                              |         |           |           |      |       |               |           |          |       |
| Total Credit |                                                           |                              |           |          |                              |         |           |           |      |       | 15            | 05        | 02       | 22    |
| Total        |                                                           | 15                           | 10        | 02       | 150                          | 350     | 150       | 25        | 25   | 700   | -             | -         | -        | -     |

SCHE  
ME



Savitribai Phule Pune University  
Second Year of Engineering (2019 Course)  
**210253: Software Engineering**

| Teaching Scheme               | Credit Scheme | Examination Scheme and Marks                                           |
|-------------------------------|---------------|------------------------------------------------------------------------|
| Lecture: <b>03 Hours/Week</b> | <b>03</b>     | Mid_Semester(TH): <b>30 Marks</b><br>End_Semester(TH): <b>70 Marks</b> |

**Prerequisite Courses :** 110005: Programming and Problem Solving

**Course Contents**

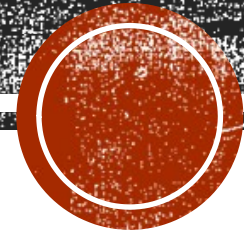
| Unit I                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Introduction to Software Engineering and Software Process Models | (06Hours) |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|-----------|
| <b>Software Engineering Fundamentals:</b> Introduction to software engineering, The Nature of Software, Defining Software, Software Engineering Practice. <b>Software Process:</b> A Generic Process Model, defining a Framework Activity, Identifying a Task Set, Process Patterns, Process Assessment and Improvement, Prescriptive Process Models, The Waterfall Model, Incremental Process Models, Evolutionary Process Models, Concurrent Models, A Final Word on Evolutionary Processes. Unified Process, Agile software development: Agile methods, plan driven and agile development. |                                                                  |           |
| <b>#Exemplar/Case Studies</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Agile Tools- JIRA                                                |           |
| <b>*Mapping of Course Outcomes for Unit I</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | CO1, CO3, CO7                                                    |           |

# SYLLA BUS



| Unit II                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Software Requirements Engineering and Analysis                                                                                                                                                                                                                                                                                                                       | (07 Hours) |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| <b>Modeling:</b> Requirements Engineering, Establishing the Groundwork, Identifying Stakeholders, Recognizing Multiple Viewpoints, working toward Collaboration, Asking the First Questions, Eliciting Requirements, Collaborative Requirements Gathering, Usage Scenarios, Elicitation Work Products, Developing Use Cases, Building the Requirements Model, Elements of the Requirements Model, Negotiating Requirements, Validating Requirements.<br><b>Suggested Free Open Source tools:</b> StarUML, Modelio, SmartDraw.                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                      |            |
| <u>#Exemplar/Case Studies</u>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Write SRS in IEEE format for selected Project Statement/ case study<br>Study SRS of Online Voting system<br>( <a href="http://dos.iitm.ac.in/OOSD_Material/CaseStudies/CaseStudy2/eVote-srs.pdf">http://dos.iitm.ac.in/OOSD_Material/CaseStudies/CaseStudy2/eVote-srs.pdf</a> ),<br>Library management System, Develop use case model for any software applications. |            |
| <u>*Mapping of Course Outcomes for Unit II</u>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | CO1, CO3, CO7                                                                                                                                                                                                                                                                                                                                                        |            |
| Unit III                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Estimation and Scheduling                                                                                                                                                                                                                                                                                                                                            | (07 Hours) |
| <b>Estimation for Software Projects:</b> The Project Planning Process, Defining Software Scope and Checking Feasibility, Resources management, Reusable Software Resources, Environmental Resources, Software Project Estimation, Decomposition Techniques, Software Sizing, Problem-Based Estimation, LOC-Based Estimation, FP-Based Estimation, Object Point (OP)-based estimation, Process-Based Estimation, Process-Based Estimation, Estimation with Use Cases, Use-Case–Based Estimation, Reconciling Estimates, Empirical Estimation Models, The Structure of Estimation Models, The COCOMO II Mode, Preparing Requirement Traceability Matrix<br><b>Project Scheduling:</b> Project Scheduling, Defining a Task for the Software Project, Scheduling.<br><b>Suggested Free Open Source Tools:</b> Gantt Project, Agantty, Project Libre. |                                                                                                                                                                                                                                                                                                                                                                      |            |
| <u>#Exemplar/Case Studies</u>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Write SRS in IEEE format for selected Project Statement/ case study, Study SRS of Online Voting system, Library management System<br>( <a href="http://dos.iitm.ac.in/OOSD_Material/CaseStudies/CaseStudy2/eVote-srs.pdf">http://dos.iitm.ac.in/OOSD_Material/CaseStudies/CaseStudy2/eVote-srs.pdf</a> ),                                                            |            |
| <u>*Mapping of Course Outcomes for Unit III</u>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | CO1, CO3, CO7                                                                                                                                                                                                                                                                                                                                                        |            |

# SYLLABUS



| Unit IV                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Design Engineering                                | (07 Hours) |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|------------|
| <p><b>Design Concepts:</b> Design within the Context of Software Engineering, The Design Process, Software Quality Guidelines and Attributes, Design Concepts - Abstraction, Architecture, design Patterns, Separation of Concerns, Modularity, Information Hiding, Functional Independence, Refinement, Aspects, Refactoring, Object-Oriented Design Concept, Design Classes, The Design Model , Data Design Elements, Architectural Design Elements, Interface Design Elements, Component-Level Design Elements, Component Level Design for Web Apps, Content Design at the Component Level, Functional Design at the Component Level, Deployment-Level Design Elements.</p> <p><b>Architectural Design:</b> Software Architecture, What is Architecture, Why is Architecture Important, Architectural Styles, A brief Taxonomy of Architectural Styles.</p> <p><b>Suggested Free Open Source Tool:</b> Smart Draw</p> |                                                   |            |
| #Exemplar/Case Studies                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Study design of Biometric Authentication software |            |
| *Mapping of Course Outcomes for Unit IV                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | CO1,CO2 CO3, CO7                                  |            |
| Unit V                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Risks and Configuration Management                | (07 Hours) |
| <p><b>Risk Management:</b> Software Risks, Risk Identification, Risk Projection, Risk Refinement, Risk Mitigation, Monitoring, and Management, The RMMM Plan.</p> <p><b>Software Configuration Management:</b> Software Configuration Management, The SCM Repository The SCM Process, Configuration Management for any suitable software system.</p> <p>Suggested Free Open Source Tools: CF Engine Configuration Tool, Puppet Configuration Tool.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                   |            |
| #Exemplar/Case Studies                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Risk management in Food delivery software         |            |
| *Mapping of Course Outcomes for Unit V                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | CO1,CO2 CO3, CO7                                  |            |

# SYLLA BUS



| Unit VI                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Software Testing                             | (07 Hours) |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|------------|
| A Strategic Approach to Software Testing, Verification and Validation, Organizing for Software Testing, Software Testing Strategy—The Big Picture, Criteria for Completion of Testing, Strategic Issues, Test Strategies for Conventional Software, Unit Testing, Integration Testing, Test Strategies for Object-Oriented Software, Unit Testing in the OO Context, Integration Testing in the OO Context, Test Strategies for Web Apps, Validation Testing, Validation-Test Criteria, Configuration Review.<br><b>Suggested Free Open Source Tools:</b> Selenium, JUnit. |                                              |            |
| <a href="#">#Exemplar/Case Studies</a>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Selenium Testing with any online application |            |
| <a href="#">*Mapping of Course Outcomes for Unit VI</a>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | CO1, CO2 CO3, CO6                            |            |

# SYLLA BUS



### Text Books:

1. Roger Pressman, "Software Engineering: A Practitioner's Approach"||, McGraw Hill, ISBN 0-07-337597-7
2. Ian Sommerville, "Software Engineering"||, Addison and Wesley, ISBN 0-13-703515-2

### Reference Books:

1. Carlo Ghezzi, "Fundamentals of Software Engineering", PHI, ISBN-10: 0133056996
2. Rajib Mall, "Fundamentals of Software Engineering"||, PHI, ISBN-13: 978-8120348981
3. Pankaj Jalote, "An Integrated Approach to Software Engineering"||, Springer, ISBN 13: 9788173192715.
4. S K Chang, "Handbook of Software Engineering and Knowledge Engineering"||, World Scientific, Vol I, II, ISBN: 978-981-02-4973-1
5. Tom Halt, "Handbook of Software Engineering", Clanye International ISBN-10: 1632402939

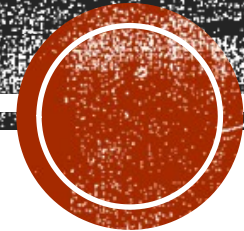
### e-books:

- <https://ebookpdf.com/roger-s-pressman-software-engineering>

### MOOC/ Video Lectures available at:

- [https://swayam.gov.in/nd1\\_noc19\\_cs69/preview](https://swayam.gov.in/nd1_noc19_cs69/preview)
- [https://swayam.gov.in/nd2\\_cec20\\_cs07/preview](https://swayam.gov.in/nd2_cec20_cs07/preview)

LEARNING  
RESOURCES



# Software Project Estimation

- Software project estimation approaches assist project managers (PMs) in effectively estimating critical project parameters such as *cost* and *scope*.
- PMs can then use these estimation strategies to give clients more accurate projections as well as budget the funds and resources they'll require for a project's success.
- Our goal is to learn—
  - ✓ which project elements should be estimated,
  - ✓ the many available projects estimating approaches, and
  - ✓ how to get started with estimation techniques that greatly help with software project estimation in software engineering.



# What is Software Project Estimation?

*Whether big or small, every project is advised to employ project estimation as a crucial step to avoid unpredictable failure in the future.*

- In short, project estimation is a complex process that revolved around predicting the time, cost, and scope that a project requires to be deemed finished.
- But in terms of software development or software engineering, it also takes the experience of the software development company, the technique they have to utilize, the process they need to follow in order to finish the project (Software Development Life Cycle).
- Project Estimation requires the use of complex tools & good mathematical as well as knowledge about planning.
- In most cases, the whole estimation process would cost the company rather considerable cost & time at the very first stage of developing a brand new website, app, or software. However, this will act as the stepping stone to make the final result more credible, realistic, and customer-satisfying.



# Which Estimations Take Place During A Project?

- There are six critical elements of a project that benefit from the use of project estimating techniques.
  - ✓ Cost,
  - ✓ Time,
  - ✓ Size or Scope,
  - ✓ Risk,
  - ✓ Resources, and
  - ✓ Quality



# Which Estimations Take Place During A Project?

- Cost:

- ✓ In project management, cost is one of the three primary constraints.
- ✓ The project will fail if you do not have sufficient funds to complete it.
- ✓ You can help set client expectations and ensure you have enough money to complete the work if you can accurately estimate project costs early on.
- ✓ Estimating costs entails determining how much money you'll need and when you'll need it.



# Which Estimations Take Place During A Project?

- Time:

- ✓ Another of the project's three main constraints is the lack of time.
- ✓ It is critical for project planning to be able to estimate both the overall project duration and the timing of individual tasks.
- ✓ You can plan for people and resources to be available when you need them if you estimate your project schedule ahead of time.
- ✓ It also enables you to manage client expectations for key deliverables.



# Which Estimations Take Place During A Project?

- Risk:

- ✓ Any unforeseen event that could positively or negatively impact your project is referred to as project risk.
- ✓ Estimating risk entails predicting what events will occur during the project's life cycle and how serious they will be.
- ✓ You can better plan for potential issues and create risk management plans if you estimate what risks could affect your project and how they will affect it.



# Which Estimations Take Place During A Project?

- Resources:

- ✓ The assets you'll need to complete the project are known as project resources.
- ✓ Tools, people, materials, subcontractors, software, and other resources are all examples of resources.
- ✓ Resource management ensures that you have all of the resources you require and make the best use of them.
- ✓ It's challenging to plan how you'll manage resources without knowing what you'll need and when. This can result in people sitting around doing nothing or materials arriving weeks after you need them.



# Which Estimations Take Place During A Project?

- Quality:

- ✓ Quality is concerned with the completion of project deliverables. Products that must adhere to stringent quality standards, such as environmental regulations, may require more money, time, and other resources than those with lower standards.
- ✓ Estimating the level of quality required by the customer aids in the planning and estimating the remaining five aspects of your project. Because all six project factors are interconnected, forecasts for one can have an impact on forecasts for the other five.



# When To Do Project Estimation In Software Engineering?

- ✓ By definition, estimating is a forecast. And according to a McKinsey report, overruns occur in 66% of all software development projects.
- ✓ This means that most of them fail to meet deadlines, go over budget, and deliver fewer features than expected.
- ✓ There could be various reasons for this, but many often think that determining a transparent software development process prevents late deliveries and budget overruns.



# Project Management Planning — Steps

- Step 1: Identify Project Stakeholders
- Step 2: Identify Project Goals and Objectives
- Step 3: Identify Project Deliverables
- Step 4: Create the Project Schedule
- Step 5: Create Supporting Plans
- Step 6: Outline the Project Plan



# Project Management Planning — Steps

- **Step 1: Identify Project Stakeholders**
- Start your project planning process by identifying the stakeholders of your project. Project stakeholders are individuals, groups, or organizations who may affect or be affected by a project. They include:
  - ✓ The project sponsor
  - ✓ The project leader
  - ✓ Project team members
  - ✓ Project testers
  - ✓ Contractors
  - ✓ Consultants
  - ✓ Customers and clients
  - ✓ Users of the project output
  - ✓ Groups impacted by the project
  - ✓ And others



# Project Management Planning — Steps

## ▪ Step 2: Identify Project Goals and Objectives

- A project's goals and objectives depend on the needs of the project stakeholders. Therefore, knowing who your stakeholders are and what their needs are is the first step in determining your project's goals. A good way to determine stakeholders' needs are stakeholder interviews, which you should conduct at the very beginning of the project planning process.
- Once you have a clear overview of your (key) stakeholders' needs, you can turn them into a set of measurable goals, following the SMART principle:
  - ✓ Specific
  - ✓ Measurable
  - ✓ Agreed
  - ✓ Realistic
  - ✓ Time-bound



# Project Management Planning — Steps

- **Step 2: Identify Project Goals and Objectives**

## Goals Vs. Objectives

- ✓ Goals are about WHAT the project needs to achieve.
- ✓ Objectives, on the other hand, are about HOW these goals can be achieved. Each goal can thus have a number of objectives.
- ✓ Example:
  - Goal:
    - ✓ Collect 500 sales qualified leads within the next three months
  - Objectives:
    - ✓ Create a white paper about the benefits of agile task management to collect marketing qualified leads
    - ✓ Set up a campaign on LinkedIn to send potential leads to the white paper landing page



# Project Management Planning — Steps

- **Step 3: Identify Project Deliverables**
  - ✓ Project deliverables are the tangible products that are produced or provided as a result of the project. We can generally distinguish between two types of deliverables:
    1. Project deliverables, such as the project plan, minutes, or reports.
    2. Product deliverables, such as intellectual material, consumer goods, contracts, and so on.



# Project Management Planning — Steps

- **Step 3: Identify Project Deliverables**

## **Deliverables vs Objectives**

- ✓ Project deliverables and project objectives are closely related, but they are not the same thing. You may need one or multiple deliverables to fulfill an objective, or you may be able to fulfill multiple objectives with just one deliverable.
- ✓ Example:
  - Objective
    - ✓ Set up a campaign on LinkedIn to send potential leads to the white paper landing page
  - Deliverables:
    - ✓ White paper landing page
    - ✓ LinkedIn ad
    - ✓ Campaign report



# Project Management Planning — Steps

- **Step 4: Create the Project Schedule**
- In traditional project management, the project schedule lists all activities and deliverables with their intended start and end dates, and thus provides a timeline for the entire project.
- To work out the schedule for your project, you will need to:
  - ✓ Define activities based on your objectives and deliverables
  - ✓ Break activities down into tasks
  - ✓ Estimate the time each task will take
  - ✓ Locate task dependencies and accommodate them in the schedule
  - ✓ Assign (human) resources to the tasks



# Project Management Planning — Steps

- **Step 4: Create the Project Schedule**
  - What is Work Breakdown Structure (WBS)?
    - ✓ A work breakdown structure (WBS) is a visual, hierarchical and deliverable-oriented deconstruction of a project.
    - ✓ It is a helpful diagram for project managers because it allows them to break down their project scope and visualize all the tasks required to complete their projects.



# Project Management Planning — Steps

- Step 4: Create the Project Schedule

## Work Breakdown Structure

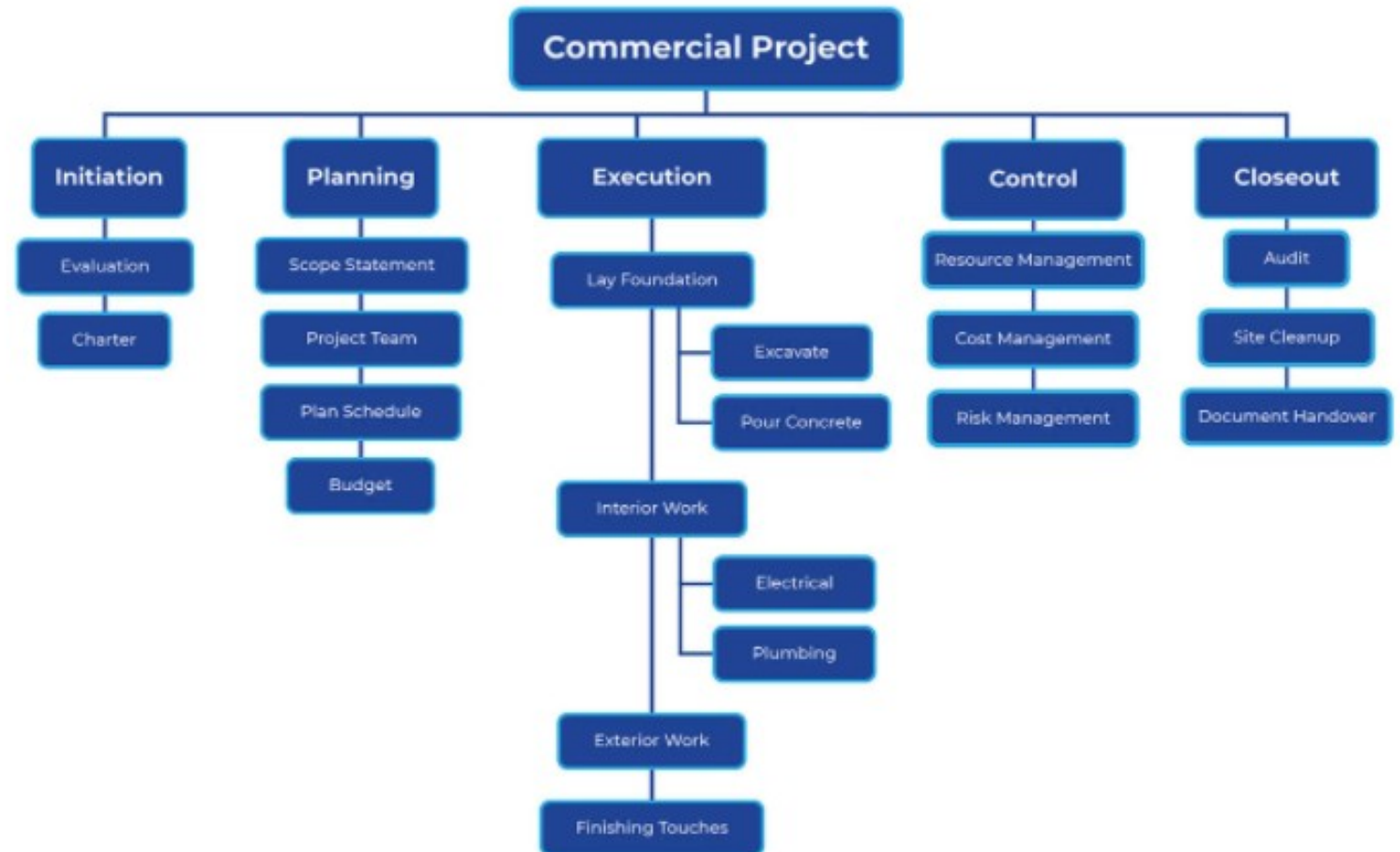


Fig. Work Breakdown Structure (WBS) Example



# Project Management Planning — Steps

- **Step 4: Create the Project Schedule**
  - Two scheduling techniques, these are network-based project management techniques
    1. Critical Path Method (CPM)
      - ✓ Activity on Arrow (AoA)
      - ✓ Activity on Node (AoN)
    2. Program (Project) Evaluation and Review Technique (PERT)



# Project Management Planning — Steps

| Activity/<br>Job | Predecessor<br>s | Duration<br>(in days) | ES | EF | LS | LF |
|------------------|------------------|-----------------------|----|----|----|----|
| A                | -                | 2                     |    |    |    |    |
| B                | -                | 3                     |    |    |    |    |
| C                | A                | 1                     |    |    |    |    |
| D                | A, B             | 4                     |    |    |    |    |
| E                | D                | 5                     |    |    |    |    |
| F                | D                | 8                     |    |    |    |    |
| G                | C, E             | 6                     |    |    |    |    |
| H                | C, E             | 4                     |    |    |    |    |
| I                | F, G, H          | 3                     |    |    |    |    |



# PERT Analysis

| Activity | Predecessors | Duration (in weeks) |       |       | $t_e$ | Variance |
|----------|--------------|---------------------|-------|-------|-------|----------|
|          |              | $t_o$               | $t_m$ | $t_p$ |       |          |
| A        | -            | 5                   | 6     | 7     |       |          |
| B        | -            | 1                   | 3     | 5     |       |          |
| C        | -            | 1                   | 4     | 7     |       |          |
| D        | A            | 1                   | 2     | 3     |       |          |
| E        | B            | 1                   | 2     | 9     |       |          |
| F        | C            | 1                   | 5     | 9     |       |          |
| G        | C            | 2                   | 2     | 8     |       |          |
| H        | E, F         | 4                   | 4     | 10    |       |          |
| I        | D            | 2                   | 5     | 8     |       |          |
| J        | H, G         | 2                   | 2     | 8     |       |          |



# PERT Analysis

| Activity | Predecessors | Duration (in weeks) |       |       | Expected Time |
|----------|--------------|---------------------|-------|-------|---------------|
|          |              | $t_o$               | $t_m$ | $t_p$ |               |
| A        | -            | 5                   | 6     | 7     | 6             |
| B        | -            | 1                   | 3     | 5     | 3             |
| C        | -            | 1                   | 4     | 7     | 4             |
| D        | A            | 1                   | 2     | 3     | 2             |
| E        | B            | 1                   | 2     | 9     | 3             |
| F        | C            | 1                   | 5     | 9     | 5             |
| G        | C            | 2                   | 2     | 8     | 3             |
| H        | E, F         | 4                   | 4     | 19    | 5             |
| I        | D            | 2                   | 5     | 8     | 5             |
| J        | H, G         | 2                   | 2     | 8     | 3             |

Calculate Total Expected Time and how long will it take to complete the project?



# The Project Planning Process

*The more you know,  
the better you estimate.  
Therefore,  
update your estimates  
as the project progresses.*

- The objective of software project planning is to provide a framework that enables the manager to make reasonable estimates of resources, cost, and schedule.
- In addition, estimates should attempt to define best-case and worst-case scenarios so that project outcomes can be bounded.
- Although there is an inherent degree of uncertainty, the software team embarks on a plan that has been established as a consequence of these tasks.
- Therefore, the plan must be adapted and updated as the project proceeds.



# Software Scope and Feasibility

*Project feasibility is important, but a consideration of business need is even more important. It does no good to build a hightech system or product that no one wants.*

- Software scope describes the functions and features that are to be delivered to end users; the data that are input and output; the “content” that is presented to users as a consequence of using the software; and the performance, constraints, interfaces, and reliability that bound the system.
- Scope is defined using one of two techniques:
  1. A narrative description of software scope is developed after communication with all stakeholders.
  2. A set of use cases is developed by end users.
- Once scope is understood, you must work to determine if it can be accomplished within the dimensions of available technology, dollars, time, and resources.
- This is a crucial, although often overlooked, part of the estimation process.



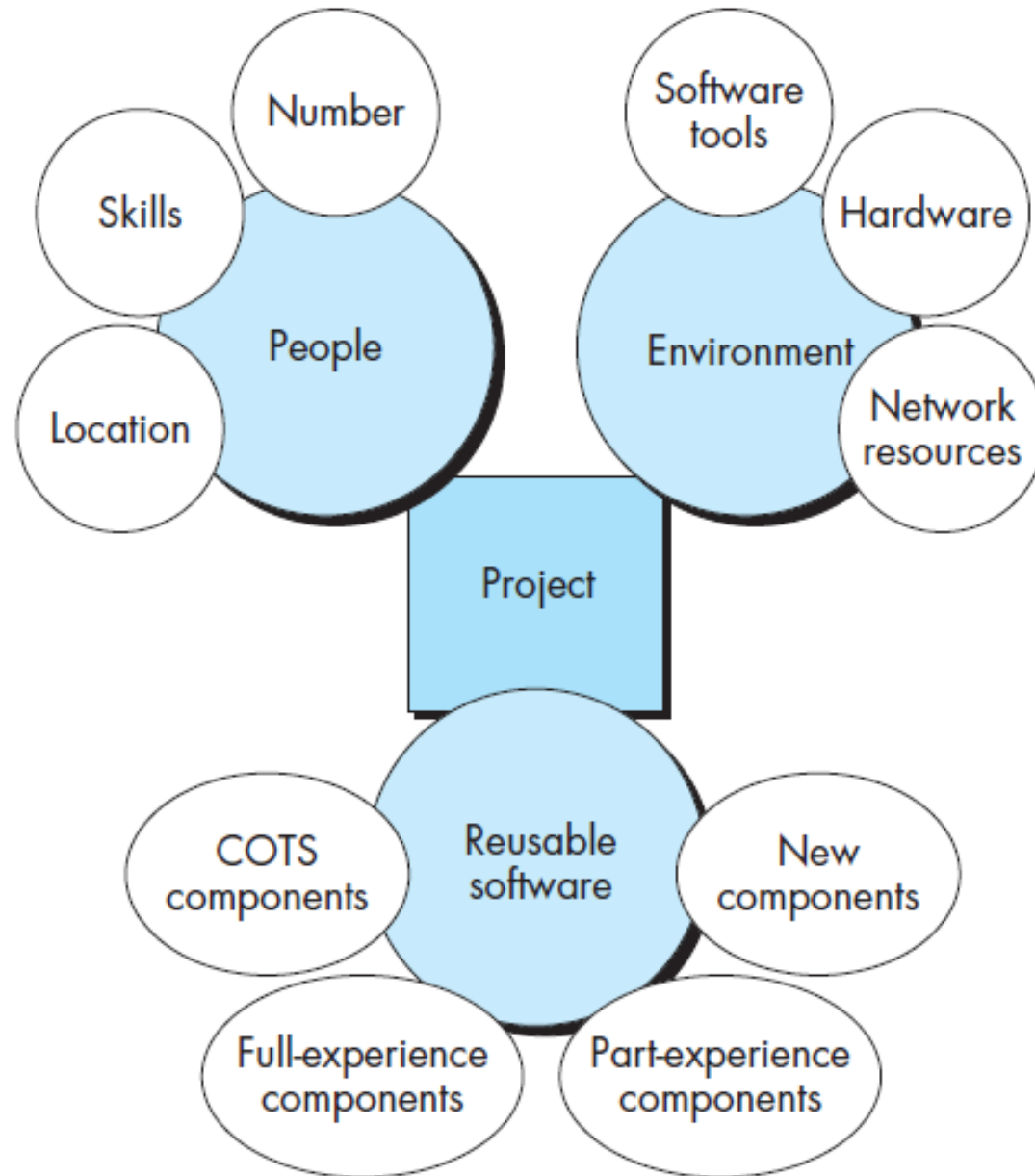
# Resources

- The second planning task is estimation of the resources required to accomplish the software development effort.
- PMs can then use these estimation strategies to give clients more accurate projections as well as budget the funds and resources they'll require for a project's success.
- Our goal is to learn—
  - ✓ which project elements should be estimated,
  - ✓ the many available projects estimating approaches, and
  - ✓ how to get started with estimation techniques that greatly help with software project estimation in software engineering.



# Resources

Fig. Project Resources



# Resources

- There are three major categories of software engineering resources—
  1. people,
  2. reusable software components, and
  3. the development environment (hardware and software tools).
- Each resource is specified with four characteristics:
  1. description of the resource,
  2. a statement of availability,
  3. time when the resource will be required, and
  4. duration of time that the resource will be applied.
- The last two characteristics can be viewed as *a time window*.



# Resources: Human Resources

- The planner begins by evaluating software scope and selecting the skills required to complete development. Both organizational position (e.g., manager, senior software engineer) and specialty (e.g., telecommunications, database, client-server) are specified.
- For relatively small projects (a few person-months), a single individual may perform all software engineering tasks, consulting with specialists as required. For larger projects, the software team may be geographically dispersed across a number of different locations. Hence, the location of each human resource is specified.

The number of people required for a software project can be determined only after an estimate of development effort (e.g., person-months) is made.



# Resources: Reusable Software Resources

- Four software resource categories that should be considered as planning proceeds:
  1. **off-the-shelf components** (existing software that can be acquired from a third party or from a past project),
  2. **full-experience components** (existing specifications, designs, code, or test data developed for past projects that are similar to the software to be built for the current project),
  3. **partial-experience components** (existing specifications, designs, code, or test data developed for past projects that are related to the software to be built for the current project but will require substantial modification), and
  4. **new components** (components built by the software team specifically for the needs of the current project).

Ironically, reusable software components are often neglected during planning.



# Resources: Reusable Software Resources

- The environment that supports a software project, often called the software engineering environment (SEE), incorporates hardware and software.
- Hardware provides a platform that supports the tools (software) required to produce the work products that are an outcome of good software engineering practice.
- Because most software organizations have multiple constituencies that require access to the SEE, you must prescribe the time window required for hardware and software and verify that these resources will be available.
- Each hardware element must be specified as part of planning.



# Software Project Estimation

- Software cost and effort estimation will never be an exact science.
- Too many variables— human, technical, environmental, political—can affect the ultimate cost of software and effort applied to develop it.
- However, software project estimation can be transformed from a black art to a series of systematic steps that provide estimates with acceptable risk.



# Software Project Estimation

- To achieve reliable cost and effort estimates, a number of options arise:
  1. Delay estimation until late in the project (obviously, we can achieve 100 percent accurate estimates after the project is complete!).
  2. Base estimates on similar projects that have already been completed.
  3. Use relatively simple decomposition techniques to generate project cost and effort estimates.
  4. Use one or more empirical models for software cost and effort estimation.



# Software Project Estimation

- To achieve reliable cost and effort estimates, a number of options arise:
  1. Delay estimation until late in the project (obviously, we can achieve 100 percent accurate estimates after the project is complete!).
  2. Base estimates on similar projects that have already been completed.
  3. Use relatively simple decomposition techniques to generate project cost and effort estimates.
  4. Use one or more empirical models for software cost and effort estimation.



# Software Project Estimation

- To achieve reliable cost and effort estimates, a number of options arise:
  1. Delay estimation until late in the project (obviously, we can achieve 100 percent accurate estimates after the project is complete!).
  2. Base estimates on similar projects that have already been completed.
  3. Use relatively simple decomposition techniques to generate project cost and effort estimates.
  4. Use one or more empirical models for software cost and effort estimation.



# Software Project Estimation

- To achieve reliable cost and effort estimates, a number of options arise:
  1. Delay estimation until late in the project (obviously, we can achieve 100 percent accurate estimates after the project is complete!).
  2. Base estimates on similar projects that have already been completed.
  3. Use relatively simple decomposition techniques to generate project cost and effort estimates.
  4. Use one or more empirical models for software cost and effort estimation.



# Software Project Estimation

- To achieve reliable cost and effort estimates, a number of options arise:
  1. Delay estimation until late in the project (obviously, we can achieve 100 percent accurate estimates after the project is complete!).
  2. Base estimates on similar projects that have already been completed.
  3. Use relatively simple decomposition techniques to generate project cost and effort estimates.
  4. Use one or more empirical models for software cost and effort estimation.



# Decomposition Techniques

- Software project estimation is a form of problem solving, and in most cases, the problem to be solved (i.e., developing a cost and effort estimate for a software project) is too complex to be considered in one piece.
- For this reason, you should decompose the problem, recharacterizing it as a set of smaller (and hopefully, more manageable) problems.
- Two different points of view:
  - i. decomposition of the problem and
  - ii. decomposition of the process.
- Estimation uses one or both forms of partitioning.
- But before an estimate can be made, we must understand the scope of the software to be built and generate an estimate of its “size.”



# Decomposition Techniques: Software Sizing

- The accuracy of a software project estimate is predicated on a number of things:

1. the degree to which you have properly estimated the size of the product to be built;
2. the ability to translate the size estimate into human effort, calendar time, and dollars (a function of the availability of reliable software metrics from past projects);
3. the degree to which the project plan reflects the abilities of the software team; and
4. the stability of product requirements and the environment that supports the software engineering effort.

The “size” of software to be built can be estimated using a direct measure, LOC, or an indirect measure, FP.



# Decomposition Techniques: Software Sizing

- If a direct approach is taken, size can be measured in lines of code (LOC).
- If an indirect approach is chosen, size is represented as function points (FP).
- Size can be estimated by considering–
  - the type of project and its application domain,
  - the functionality delivered (i.e., the number of function points),
  - the number of components to be delivered,
  - the degree to which a set of existing components must be modified for the new system.



# Decomposition Techniques:

## Problem-Based estimation

- Lines of code (LOC) and function points (FP) can be described as measures from which productivity metrics can be computed.
- LOC and FP data are used in two ways during software project estimation:
  1. as estimation variables to “size” each element of the software and
  2. as baseline metrics collected from past projects and used in conjunction with estimation variables to develop cost and effort projections.



# Decomposition Techniques:

## Problem-Based estimation

- LOC and FP estimation are distinct estimation techniques.
- Yet both have a number of characteristics in common.
- You begin with a bounded statement of software scope and from this statement attempt to decompose the statement of scope into problem functions that can each be estimated individually.
- LOC or FP (the estimation variable) is then estimated for each function. Alternatively, you may choose another component for sizing, such as classes or objects, changes, or business processes affected.
- Baseline productivity metrics (e.g., LOC/pm or FP/pm) are then applied to the appropriate estimation variable, and cost or effort for the function is derived.
- Function estimates are combined to produce an overall estimate for the entire project.



# Decomposition Techniques:

## Problem-Based estimation

- Making the use of a single-baseline productivity metric is not suggested.
- In general, LOC/pm or FP/pm averages should be computed by project domain.
- That is, projects should be grouped by team size, application area, complexity, and other relevant parameters. Local domain averages should then be computed.
- When a new project is estimated, it should first be allocated to a domain, and then the appropriate domain average for productivity should be used in generating the estimate.



# Decomposition Techniques:

## LOC-Based Estimation

- As an example of LOC and FP problem-based estimation techniques, we consider a software package to be developed for a computer-aided design application for mechanical components.
- The software is to execute on a desktop workstation and must interface with various computer graphics peripherals including a mouse, digitizer, high-resolution color display, and laser printer. A preliminary statement of software scope can be developed:

The mechanical CAD software will accept two- and three-dimensional geometric data from a designer. The designer will interact and control the CAD system through a user interface that will exhibit characteristics of good human/machine interface design. All geometric data and other supporting information will be maintained in a CAD database. Design analysis modules will be developed to produce the required output, which will be displayed on a variety of devices. The software will be designed to control and interact with peripheral devices that include a mouse, scanner, laser printer, and plotter.

# Decomposition Techniques:

## LOC-Based Estimation

- This statement of scope is preliminary—it is not bounded. Every sentence would have to be expanded to provide concrete detail and quantitative bounding.

| Function                                     | Estimated LOC |
|----------------------------------------------|---------------|
| User interface and control facilities (UICF) | 2,300         |
| Two-dimensional geometric analysis (2DGA)    | 5,300         |
| Three-dimensional geometric analysis (3DGA)  | 6,800         |
| Database management (DBM)                    | 3,350         |
| Computer graphics display facilities (CGDF)  | 4,950         |
| Peripheral control function (PCF)            | 2,100         |
| Design analysis modules (DAM)                | 8,400         |
| <i>Estimated lines of code</i>               | <i>33,200</i> |

Assume that further refinement has occurred and that the major software functions listed in Figure are identified. Following the decomposition technique for LOC, an estimation table is developed. A range of LOC estimates is developed for each function. For example, the range of LOC estimates for the 3D geometric analysis function is optimistic, 4600 LOC; most likely, 6900 LOC; and pessimistic, 8600 LOC.

**Fig. Estimation table for the LOC methods**



# Decomposition Techniques:

## LOC-Based Estimation

- A review of historical data indicates that the organizational average productivity for systems of this type is 620 LOC/pm.
- Based on a burdened labor rate of \$8,000 per month, the cost per line of code is approximately \$13.
- Based on the LOC estimate and the historical productivity data, the total estimated project cost is \$431,000 and the estimated effort is 54 person-months.



# Decomposition Techniques:

## Function Point (FP)-Based Estimation

- The function point (FP) metric can be used effectively as a means for measuring the functionality delivered by a system.
- Using historical data, the FP metric can then be used to
  1. estimate the cost or effort required to design, code, and test the software;
  2. predict the number of errors that will be encountered during testing; and
  3. forecast the number of components and/or the number of projected source lines in the implemented system.



# Decomposition Techniques:

## Function Point (FP)-Based Estimation

|                                |                                                                              |
|--------------------------------|------------------------------------------------------------------------------|
| External Inputs (EIs):         | Functions related to data entering the system.                               |
| External outputs (EOs):        | Functions related to data exiting the system.                                |
| External Inquiries (EQs):      | They lead to data retrieval from the system but don't change the system.     |
| Internal Logical Files (ILFs): | Logical files maintained within the system. Log files are not included here. |
| External Interface             | These are logical files for other                                            |



# Decomposition Techniques:

## Function Point (FP)-Based Estimation

|                                |                                                                              |
|--------------------------------|------------------------------------------------------------------------------|
| External Inputs (EIs):         | Functions related to data entering the system.                               |
| External outputs (EOs):        | Functions related to data exiting the system.                                |
| External Inquiries (EQs):      | They lead to data retrieval from the system but don't change the system.     |
| Internal Logical Files (ILFs): | Logical files maintained within the system. Log files are not included here. |
| External Interface             | These are logical files for other                                            |



# Decomposition Techniques:

## Function Point (FP)-Based Estimation

- The function point (FP) metric can be used effectively as a means for measuring the functionality delivered by a system.

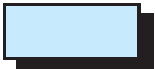
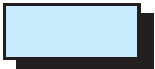
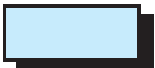
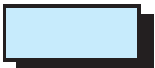
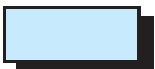
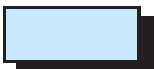
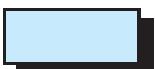
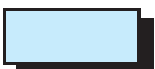
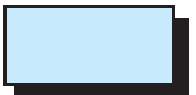
| Information Domain Value        | Count                                                                                | Weighting factor |         |         |    |                                                                                         |
|---------------------------------|--------------------------------------------------------------------------------------|------------------|---------|---------|----|-----------------------------------------------------------------------------------------|
|                                 |                                                                                      | Simple           | Average | Complex |    |                                                                                         |
| External Inputs (EIs)           |    | 3                | 3       | 4       | 6  | =    |
| External Outputs (EOs)          |    | 3                | 4       | 5       | 7  | =    |
| External Inquiries (EQs)        |    | 3                | 3       | 4       | 6  | =    |
| Internal Logical Files (ILFs)   |   | 3                | 7       | 10      | 15 | =   |
| External Interface Files (EIFs) |  | 3                | 5       | 7       | 10 | =  |
| Count total                     |  |                  |         |         |    |    |

Fig. Computing function points



# Decomposition Techniques:

## Function Point (FP)-Based Estimation

- Consider a simple flow diagram for a user interaction function within SafeHome software, represented in Figure

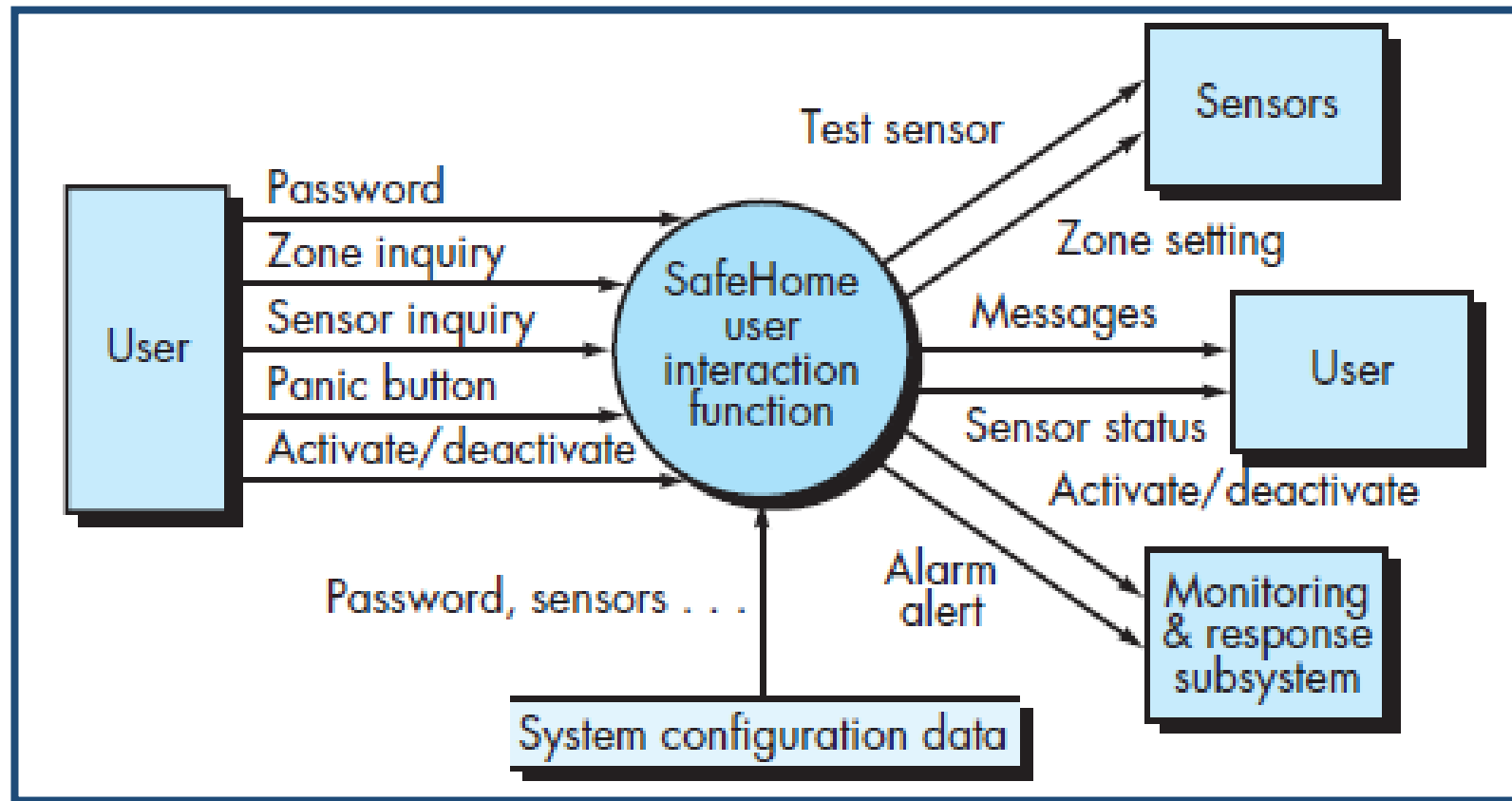


Fig. A flow model



# Decomposition Techniques:

## Function Point (FP)-Based Estimation

- The function point (FP) metric can be used effectively as a means for measuring the functionality delivered by a system.

| Information Domain Value        | Count |   | Weighting factor |         |         |   |    |
|---------------------------------|-------|---|------------------|---------|---------|---|----|
|                                 |       |   | Simple           | Average | Complex | = |    |
| External Inputs (EIs)           | 3     | 3 | 3                | 4       | 6       | = | 9  |
| External Outputs (EOs)          | 2     | 3 | 4                | 5       | 7       | = | 8  |
| External Inquiries (EQs)        | 2     | 3 | 3                | 4       | 6       | = | 6  |
| Internal Logical Files (ILFs)   | 1     | 3 | 7                | 10      | 15      | = | 7  |
| External Interface Files (EIFs) | 4     | 3 | 5                | 7       | 10      | = | 20 |
| Count total                     |       |   |                  |         |         |   | 50 |

Fig. Computing function points



# Decomposition Techniques:

## Function Point (FP)-Based Estimation

- To compute function points (FP), the following relationship is used:

$$FP = \text{count total} \times [0.65 + 0.01 \times \Sigma(F_i)]$$

- The  $F_i$  ( $i = 1$  to 14) are *value adjustment factors* (VAF) based on responses to the 14 questions.



# Decomposition Techniques:

## Function Point (FP)-Based Estimation

1. Does the system require reliable backup and recovery?
2. Are specialized data communications required to transfer information to or from the application?
3. Are there distributed processing functions?
4. Is performance critical?
5. Will the system run in an existing, heavily utilized operational environment?
6. Does the system require online data entry?
7. Does the online data entry require the input transaction to be built over multiple screens or operations?
8. Are the ILFs updated online?
9. Are the inputs, outputs, files, or inquiries complex?
10. Is the internal processing complex?
11. Is the code designed to be reusable?
12. Are conversion and installation included in the design?
13. Is the system designed for multiple installations in different organizations?
14. Is the application designed to facilitate change and ease of use by the user?



# Decomposition Techniques:

## Function Point (FP)-Based Estimation

- Assume that past data indicates that one FP translates into 60 lines of code (an object-oriented language is to be used) and that 12 FPs are produced for each person-month of effort.
- These historical data provide the project manager with important planning information that is based on the requirements model rather than preliminary estimates.
- Assume further that past projects have found an average of three errors per function point during requirements and design reviews and four errors per function point during unit and integration testing.
- These data can ultimately help you assess the completeness of your review and testing activities.



# Decomposition Techniques:

## Function Point (FP)-Based Estimation

### EXERCISE:

- A system has 12 external inputs, 24 external outputs, fields 30 different external queries, manages 4 internal logical files, and interfaces with 6 different legacy systems (6 EIFs).
- All of these data are of average complexity and the overall system is relatively simple. Compute FP for the system.



# Decomposition Techniques:

## Process-Based Estimation

| Activity<br>→ | CC   | Planning | Risk<br>analysis | Engineering |        | Construction<br>release |       | CE  | Totals |
|---------------|------|----------|------------------|-------------|--------|-------------------------|-------|-----|--------|
| Task →        |      |          |                  | Analysis    | Design | Code                    | Test  |     |        |
| Function<br>↓ |      |          |                  |             |        |                         |       |     |        |
| UICF          |      |          |                  | 0.50        | 2.50   | 0.40                    | 5.00  | n/a | 8.40   |
| 2DGA          |      |          |                  | 0.75        | 4.00   | 0.60                    | 2.00  | n/a | 7.35   |
| 3DGA          |      |          |                  | 0.50        | 4.00   | 1.00                    | 3.00  | n/a | 8.50   |
| CGDF          |      |          |                  | 0.50        | 3.00   | 1.00                    | 1.50  | n/a | 6.00   |
| DBM           |      |          |                  | 0.50        | 3.00   | 0.75                    | 1.50  | n/a | 5.75   |
| PCF           |      |          |                  | 0.25        | 2.00   | 0.50                    | 1.50  | n/a | 4.25   |
| DAM           |      |          |                  | 0.50        | 2.00   | 0.50                    | 2.00  | n/a | 5.00   |
|               |      |          |                  |             |        |                         |       |     |        |
|               |      |          |                  |             |        |                         |       |     |        |
| Totals        | 0.25 | 0.25     | 0.25             | 3.50        | 20.50  | 4.50                    | 16.50 |     | 46.00  |
|               |      |          |                  |             |        |                         |       |     |        |
| % effort      | 1%   | 1%       | 1%               | 8%          | 45%    | 10%                     | 36%   |     |        |

CC – customer communication CE – customer evaluation

Fig. Process-based estimation table

- The most common technique for estimating a project is to base the estimate on the process that will be used.
- That is, the process is decomposed into a relatively small set of activities, actions, and tasks and the effort required to accomplish each is estimated.
- Like the problem-based techniques, process-based estimation begins with a depiction of software functions obtained from the project scope.
- A series of framework activities must be performed for each function.
- Functions and related framework activities may be represented as part of a table



# Decomposition Techniques:

## Process-Based Estimation

- Once problem functions and process activities are blended, you estimate the effort (e.g., person-months) that will be required to accomplish each software process activity for each software function. These data constitute the central matrix of the table.
- Average labor rates (i.e., cost/unit effort) are then applied to the effort estimated for each process activity.

If process-based estimation is performed independently of LOC or FP estimation, you now have two or three estimates for cost and effort that may be compared and reconciled.

If both sets of estimates show reasonable agreement, there is good reason to believe that the estimates are reliable otherwise further analysis is required.



# Decomposition Techniques:

## Estimation with Use cases

- Use cases provide a software team with insight into software scope and requirements.
- Once use cases have been developed, they can be used to estimate the projected “size” of a software project. However, developing an estimation approach with use cases presents challenges.
- Use cases are described using many different formats and styles and represent an external view (the user’s view) of the software. Therefore, they can be written at many different levels of abstraction.
- Use cases do not address the complexity of the functions and features that are described, and they can describe complex behavior (e.g., interactions) that involve many functions and features.



# Decomposition Techniques:

## Estimation with Use cases

- Even with these constraints, it is possible to compute use case points (UCPs) in a manner that is analogous to the computation of functions points.
- The computation of use case points must take the following characteristics into account:
  - ✓ The number and complexity of the use cases in the system.
  - ✓ The number and complexity of the actors on the system.
  - ✓ Various nonfunctional requirements (such as portability, performance, maintainability) that are not written as use cases.
  - ✓ The environment in which the project will be developed (e.g., the programming language, the software team's motivation).



# Decomposition Techniques:

## Estimation with Use cases

### STEP-1

How to  
determine  
the  
number  
and  
complexit  
y of the  
use cases  
in the

- To begin, each use case is assessed to determine its relative complexity.
- A simple use case indicates a simple user interface, a single database, and three or fewer transactions and five or fewer class implementations.
- An average use case indicates a more complex UI, 2 or 3 databases, and 4 to 7 transactions with 5 to 10 classes.
- Finally, a complex use case implies a complex UI with multiple databases, using eight or more transactions and 11 or more classes.
- Each use case is assessed using these criteria and the count of each type is weighted by a factor of 5, 10, and 15, respectively.
- A total unadjusted use case weight (UUCW) is the sum of



# Decomposition Techniques:

## Estimation with Use cases

### STEP-2

How to  
determine  
the  
number  
and  
complexit  
y of the  
actors on  
the

- Next, each actor is assessed.
- Simple actors are automations (another system, a machine or device) that communicate through an API.
- Average actors are automations that communicate through a protocol or a data store, and
- Complex actors are humans who communicate through a GUI or other human interface.
- Each actor is assessed using these criteria and the count of each type is weighted by a factor of 1, 2, and 3, respectively.
- The total unadjusted actor weight (UAW) is the sum of all weighted counts.



# Decomposition Techniques:

## Estimation with Use cases

- These unadjusted values are modified by considering technical complexity factors (TCFs) and environment complexity factors (ECFs).
- Thirteen factors contribute to an assessment of the final TCF, and eight factors contribute to the computation of the final ECF.
- Once these values have been determined, the final UCP value is computed in the following manner:

$$UCP = (UUCW + UAW) \times TCF \times ECF$$



# Decomposition Techniques:

## Estimation with Use cases: *An Example*

- Sixteen complex use cases describe the user interface subsystem. The engineering subsystem group is described by 14 average use cases and 8 simple use cases. And the infrastructure subsystem is described with 10 simple use cases.
- Analysis of the use cases indicates that there are 8 simple actors, 12 average actors, and 4 complex actors.
- After evaluation of the technology and the environment,  $TCF = 1.04$  and  $ECF = 0.96$ .
- Compute Use Case Points (UCP).



# Decomposition Techniques:

## Estimation with Use cases: *An Example*

- Sixteen complex use cases describe the user interface subsystem. The engineering subsystem group is described by 14 average use cases and 8 simple use cases. And the infrastructure subsystem is described with 10 simple use cases.

- Therefore,

$$\begin{aligned} \text{UUCW} &= (16 \text{ use cases} \times 15) + [(14 \text{ use cases} \times 10) \\ &\quad + (8 \text{ use cases} \times 5)] + (10 \text{ use cases} \times 5) \\ &= 470 \end{aligned}$$



# Decomposition Techniques:

## Estimation with Use cases: *An Example*

- Analysis of the use cases indicates that there are 8 simple actors, 12 average actors, and 4 complex actors.

- Therefore,

$$\text{UAW} = (8 \text{ actors} \times 1) + (12 \text{ actors} \times 2) + (4 \text{ actors} \times 3)$$

$$= 44$$



# Decomposition Techniques:

## Estimation with Use cases: *An Example*

- After evaluation of the technology and the environment, TCF = 1.04 and ECF = 0.96.
- Compute Use Case Points (UCP).

Since  $UCP = (UUCW + UAW) \times TCF \times ECF$

$$UCP = (470 + 44) \times 1.04 \times 0.96$$
$$= 513$$

Using past project data as a guide, the development group has produced 85 LOC per UCP.

Therefore, an estimate of the overall size of the CAD project is 43,600 LOC.

Similar computations can be made for applied effort or project

# Decomposition Techniques: Reconciling Estimates

- The estimation techniques discussed so far result in multiple estimates that must be reconciled to produce a single estimate of effort, project duration, or cost.
- What happens when agreement between estimates is poor? The answer to this question requires a reevaluation of information used to make the estimates.
- Widely divergent estimates can often be traced to one of two causes:
  - 1) the scope of the project is not adequately understood or has been misinterpreted by the planner, or
  - 2) productivity data used for problem-based estimation techniques is inappropriate for the application, obsolete (in that it no longer accurately reflects the software engineering organization), or has been misapplied. You should determine the cause of divergence and then reconcile the estimates.



# Empirical Estimation Models

- An estimation model for computer software uses empirically derived formulas to predict effort as a function of LOC or FP.
- We also know the approach to estimate LOC or FP. Here, the resultant values for LOC or FP are plugged into the estimation model.



# Empirical Estimation Models

## The Structure of Estimation Models

- A typical estimation model is derived using regression analysis on data collected from past software projects. The overall structure of such models takes the form

$$E = A + B \times (e_v)^C$$

where-

$A$ ,  $B$ , and  $C$  are empirically derived constants,

$E$  is effort in person-months, and

$e_v$  is the estimation variable (either LOC or FP).



# Empirical Estimation Models

## The COCOMO II Model

- COCOMO - COnstructive COst MOdel
- The original COCOMO model became one of the most widely used and discussed software cost estimation models in the industry.
- It has evolved into a more comprehensive estimation model, called COCOMO II. Like its predecessor, COCOMO II is actually a hierarchy of estimation models that address different “stages” of the software process.
- Like all estimation models for software, the COCOMO II models require sizing information. Three different sizing options are available as part of the model hierarchy: object points, function points, and lines of source code.



# Empirical Estimation Models

## The COCOMO II Model

- COCOMO - COnstructive COst MOdel
- COCOMO II is actually a hierarchy of estimation models that address the following areas:
- Application composition model- Used during the early stages of software engineering.
- Early



# Empirical Estimation Models

## The COCOMO II Model

- The Software Equation:
- The software equation is a dynamic multivariable model that assumes a specific distribution of effort over the life of a software development project.
- The model has been derived from productivity data collected for over 4,000 contemporary software projects.

where,

***E*** = effort in person-months or person-years

***t*** = 
$$E = \frac{LOC \times B^{0.333}}{P^3} \times \frac{1}{t^4}$$
 months or years

***B*** = “special skills factor”

***P*** = “productivity parameter”



# Empirical Estimation Models

## The COCOMO II Model

- The Software Equation:

$$E = \frac{LOC \times B^{0.333}}{P^3} \times \frac{1}{t^4}$$

- Typical values might be,  
 $P = 2,000$  for development of real-time embedded software,  
 $P = 10,000$  for telecommunication and systems software, and  
 $P = 28,000$  for business systems applications.  
The productivity parameter can be derived for local conditions using historical data collected from past development efforts.



# Empirical Estimation Models

## The COCOMO II Model

- The Software Equation 
$$E = \frac{LOC \times B^{0.333}}{P^3} \times \frac{1}{t^4}$$

- The software equation has two independent parameters:

1. an estimate of size (in LOC) and
2. an indication of project duration in calendar months or years.

- To simplify the estimation process and use a more common form for estimation model, a set of equations derived from the software equation are suggested.

- Minimum development time is defined as

- Therefore

$$t_{\min} = 8.14 \frac{LOC}{P^{0.43}} \text{ in months for } t_{\min} > 6 \text{ months}$$

- Ex. with  $P = 12,000$

$$E = 180 B t^3 \text{ in person-months for } E \geq 20 \text{ person-months}$$

$$t_{\min} = 8.14 \times \frac{33,200}{12,000^{0.43}} = 12.6 \text{ calendar months}$$

$$E = 180 \times 0.28 \times (1.05)^3 = 58 \text{ person-months}$$